WASP Software Engineering Course Assignment 2

Yi Peng

August 22, 2026

1 Introduction

The adoption of Artificial Intelligence (AI) solutions has surged across numerous sectors; however, a remarkably high number of these software solutions either fail to reach production or fail post-deployment [6]. Because there is no AI without data [5], these failures rarely stem from the machine learning models themselves. Instead, they originate from pervasive data issues, including poor quality, unrepresentative samples, and unexpected shifts in data over time. Managing these data characteristics requires collaboration among diverse roles [13, 9], making it a Requirements Engineering (RE) challenge.

Despite the need for robust data management throughout the AI lifecycle, a significant disconnect exists between the established methodologies of the RE community and the emerging practices of the broader AI Engineering (AIE) landscape. My research focuses on Data Requirements for AI/ML Systems, aiming to bridge the integration gap between RE specification and AIE execution. I will investigate how operational friction in AI development often stems from the absence of upfront data specification and negotiation. Furthermore, I want to develop frameworks that translate data requirements (such as completeness, fairness, and explainability) into machine-interpretable constraints for automated pipelines. By linking requirements elicitation to runtime validation, my research aims to establish practices that manage data continuously, enabling the development of verifiable and reliable AI systems.

2 Lecture Principles

2.1 Technical Debt

Both Sculley et al. [10] and Bogner et al. [2] established that data is the most dangerous source of technical debt in machine learning. The former argued that data dependencies cost more than code dependencies because they create unstable, hidden, and compounding risks. Reinforcing this, Bogner et al. identified "data debt" as the most recurrent new type of technical debt in AI systems, originating from poor data collection, management, and relevance.

The operational failures described in both papers can be seen as symptoms of missing data requirements. Bogner et al. finds that the majority of AI antipatterns stem from data and model deficiencies, citing issues like missing metadata, data duplication, and miscoding. Similarly, Sculley et al. highlights "pipeline jungles" and "unstable data dependencies" as system-level antipatterns that cripple long-term maintenance. These issues occur because data properties are assumed rather than explicitly elicited and specified.

My research on data requirements for AI/ML systems can serve as the preventative layer for technical debt. AI practitioners currently accrue data debt because they lack the RE frameworks

needed to specify data constraints upfront. By defining requirements for data quality, provenance, and dependencies, my research aims to provide the tools to prevent the "data debt" characterized by [2] and mitigate the data dependencies warned against by [10].

2.2 Behavioral Software Engineering

Specifically, I want to look at cognitive biases, and more recently, the fact that software developers are incorporating more generative AI into their work. Human cognitive limitations fundamentally threaten the integrity of AI/ML systems by encoding historical and social inequities directly into datasets and algorithms. As [12, 11] demonstrates, cognitive biases like "similar-to-me" and stereotype biases frequently pollute the historical data used to train models. Furthermore, domain experts often rely on inappropriate assumptions about job functions and soft skills, which leads developers to choose suboptimal predictive variables and formulate algorithms based on missing or inadequate information [12]. This shows that without data requirements, the cognitive blind spots of human decision-makers become embedded in the AI system.

The introduction of agentic AI coding assistants further amplifies these risks. According to [1], developers interacting with tools like GitHub Copilot often operate in an "acceleration mode", rapidly accepting AI code suggestions through superficial visual pattern matching rather than engaging in thorough validation. When faced with unfamiliar tasks, developers shift into an "exploration mode", heavily relying on the AI to drive the code structure, which ultimately leaves them with generated code that is difficult to debug. If developers use this accelerated, AI-driven workflow to build data pipelines without explicitly defined data requirements, they risk hardcoding their own cognitive biases. This also creates the "data debt" and unstable dependencies that [10] and [2] identify as the root causes of long-term AI system failure.

Soleimani et al. [11] proposed a cross-functional knowledge-sharing framework that aligns with the need for explicit data requirements. During pre-development, domain experts must share their knowledge to define objective criteria so developers know precisely how to model the data. Throughout development, these experts must actively guide data labeling to reduce bias and provide accurate context for machine learning algorithms. Post-development also requires experts to test model outputs and provide continuous feedback so developers can iteratively retrain the algorithms. It might be possible that by enforcing these negotiated data requirements, the speed and efficiency gained from agentic AI tools do not come at the cost of biased machine learning systems.

3 Guest Lecture Principles

3.1 Formal RE+LLMs: A Two-way Roadmap

The integration of Formal Methods (FMs) and Large Language Models (LLMs) proposed by [3] offers a mechanism to enforce data requirements. Ferrari and Spoletini [3] outlined a two-way roadmap where LLMs are used to make complex formal methods more accessible, while FMs are applied to LLMs to guarantee correctness, mitigate hallucinations, and ensure trustworthiness. Because LLMs possess weak inference abilities and struggle with logical coherence, FMs provide the mathematical boundaries needed to control AI behavior and outputs.

This provides a possibility to mathematically verify that data requirements are met. As human cognitive biases easily pollute training datasets and feature selection, using FMs to formally specify data requirements, like finding a way to formalize ethical requirements to mitigate stereotype biases, creates verifiable rules for the system. Furthermore, Ferrari and Spoletini [3] highlight the use of formal ontologies and knowledge graphs to inject structured domain knowledge into AI systems.

Data requirements could be defined through formal domain ontologies, and it might be possible to explicitly constrain the AI's reasoning process and prevent the system from relying on the inappropriate human assumptions that typically lead to biased algorithms.

Ferrari and Spoletini [3] also introduced "formal prompt engineering", which can be used to address the risks of developers over-relying on agentic AI coding assistants. When developers use AI to build data collection or processing pipelines, they can embed formal pre- and post-conditions directly into their prompts. This approach ensures that the generated pipeline code adheres to negotiated data requirements.

3.2 Goal Modeling

Goal modeling has been researched quite extensively in RE for software engineering of non-AI based systems [15]. But for AI/ML systems specifically, the field is still in its early stages. The most recent being [8], which demonstrated how AI techniques like deep learning and logical rules can semi-automate the creation of traditional iStar goal models from natural language requirements, but conventional goal models remain limited in capturing the inductive, data-centric dependencies of ML systems. Thus, goal modeling still needs some extensions to address these limitations.

Directly aligning to my topic, Yamani et al. [16] introduced the Data Requirement Goal Model using the Goal-oriented Requirement Language to elicit, structure, and evaluate data requirements prior to model training. They demonstrated that data challenges can be formalized as softgoals and key performance indicators to evaluate alternative datasets against system goals, preventing the downstream "garbage-in, garbage-out" failures common in ML development.

Traveter et al. [14] addresses the socio-technical dimension of data requirements by extending motivational goal models to elicit and model emotional goals and human values such as fairness, benevolence, and trust. Because training datasets often reflect and amplify human prejudices, they show that grounding system requirements in human values early in the design phase ensures that data selection and labeling criteria align with ethical and societal expectations.

I think these studies show that goal modeling provides abstractions that can be used to specify and evaluate data requirements for AI/ML systems.

4 Data scientists, software engineers, and AI engineers

According to the Introduction chapter, data scientists typically have educational backgrounds in statistics and machine learning, and they focus primarily on building models, gathering data, and using exploratory workflows in computational notebooks to achieve high accuracy on test datasets. The From Models to Systems chapter adds that data scientists are accustomed to working in local workspaces and often resent having to do infrastructure work that prevents them from focusing on their models. On the other hand, the Introduction chapter describes software engineers as focusing on delivering complete, functional software products that satisfy user needs while navigating strict budget and time constraints.

I mostly agree with the chapters, but the gap between these roles is shrinking. Software engineers are learning a lot more about machine learning. A few years ago, a software interviewer might not understand reinforcement learning, but today, almost all software engineers know the basics of ML. On the other hand, many data scientists still do not know or care about software practices like requirements engineering. However, the data science mindset is changing too. There is more critique against just "SOTA chasing", and major ML conferences like NeurIPS feature papers about technical debt. This shows that both sides are starting to care more about building a complete, working system rather than just focusing on their own parts.

The argument about production ML requiring a system-wide view is very relevant to my research in data requirements for AI/ML systems. A model-centric view would wrongly assume that data is just handed over and remains stable. However, data requirements consider a system-wide view and go beyond just deciding what data to use. The entire infrastructure surrounding that data needs to be taken into account. For example, consider how the user interface is designed to collect data, the large-scale data management pipelines needed to process it, and how the system is updated when real-world data changes over time. Therefore, defining data requirements means planning for how the whole system interacts with and maintains that data, not just what the model needs to train and test.

I believe data scientists and software engineers will remain distinct roles, but everyone needs to learn a bit about the other's domain. As the Introduction chapter points out, building a production ML system requires too many different skills for one person to be an expert in everything. However, the whole system only works if people have a general idea of what the other side is doing. Many AI projects fail because people have incorrect or overly optimistic views about what AI can actually do. Therefore, we still need distinct experts, but we also need people who can stand in the middle, maybe a managerial role, who can communicate effectively across both sides and keep expectations grounded.

I personally think AI engineering is the middle ground. An AI engineer is someone who knows machine learning but also has some software engineering training. This means they care about the performance of the model, but they also care about how the system as a whole is actually built, deployed, and maintained over time. The From Models to Systems chapter actually notes that the "AI Engineering" or "ML engineer" title is becoming popular to describe this exact focus on creating ML infrastructure.

I think new AI tools make it easier for people in one role to learn a bit about the others. As the From Models to Systems chapter notes, foundation models and RAG systems are changing the workflow. However, the boundaries between the roles have not disappeared because humans still need to verify the AI's output and fix mistakes. Nothing good happens when people blindly trust AI output. The fundamental jobs remain the same: a data person still has to find good data, a software engineer still has to negotiate requirements and run tests, and an AI engineer still has to oversee the whole system to ensure it actually works. Even though the Introduction chapter mentions that large language models change some trade-offs and introduce new security issues, it also confirms that engineering tasks like testing and hazard analysis are just as important as ever. Even with new multi-agent systems, a real person in each role supervising the work is still needed.

5 Paper Analysis

5.1 RegDriftKit: A Toolkit for Generating Data and Benchmarking Drift Detection in Regression Tasks [7]

The core idea of the paper is the development of RegDriftKit, a framework designed to generate synthetic datasets, monitor concept drift and benchmark drift detection algorithms for regression tasks. It combines different drift detection programs into one shared format. This makes it easy to use and compare different methods side by side. From a SE perspective, this tool makes AI systems easier to test and maintain. In production AI engineering, engineers usually know a problem exists when the AI starts making bad predictions. By generating explicit drift ground truths, the toolkit allows engineers to systematically fine tune drift detectors independently prior to deployment. The shared format also allows software teams to easily swap out detection tools without breaking the rest of their software pipeline.

In relation to my research, this paper directly supports RE by providing a way to define and test data shifts. In AI systems, a major challenge is writing clear requirements for how a system should handle changing data. The toolkit lets engineers specify data requirements using a target p-value. The paper also introduces ways to measure detection speed and accuracy using metrics like the Mean Time Ratio. These metrics can be used to write non-functional requirements for data monitors. For example, a requirement can state that a data monitor must catch a specific type of data drift within a set timeframe without triggering false alarms. The unified interface of the toolkit then allows testing different tools to see which one meets these requirements. To prove that an AI system meets its data requirements, the right test data is needed. Real data often lacks clear labels showing exactly when a shift happened. The toolkit can help create artificial data that includes known data drift points. It is then possible to build validation datasets that prove the AI system operates correctly under different data conditions.

The AI system I will use here is based on the original paper. It is a regression system that is used by a large business to manage their physical equipment. By looking at past data such as temperature readings, vibration levels, and usage hours the system predicts remaining days a machine can operate safely before it fails. The tool can be used to create synthetic datasets that mirror actual equipment sensor data. By inputting a target p-value, the software automatically creates test scenarios where the data shifts over time, simulating hardware degradation or changing operating conditions. Once the test data is generated, the toolkit provides a unified interface for existing detection libraries, allowing the user to plug in different monitoring tools and measure how quickly and accurately they flag the simulated data shifts. The toolkit also provides a retraining pipeline that can be used to observe how the AI system behaves when these automated alerts trigger a model update.

For this system, my research in data requirements will first start with defining a process that translates high-level operational goals into specific data constraints. Exact data input needs to be established, so that every data attribute collected directly supports the predictive function of the software. Gathering these requirements means structured communication with domain experts and business stakeholders. Historical data the enterprise currently stores can be used to determine what operational variables are considered highly predictive of mechanical wear. Employing standardized requirements templates helps organize these conversations. This systematic elicitation prevents gaps that could later degrade the performance of the regression models. Metrics must also be defined for each data requirement so that they are testable.

Traditional software requirements assume static operational environments, but machine learning systems degrade as real-world data drifts away from the original training distribution. I think this paper shows that data requirements should also consider data drift tolerances. In addition to defining what data must be collected, the requirements must specify acceptable statistical boundaries for the incoming data streams over time. From another angle, requirements evolution for data requirements is needed as well. Change management frameworks specifically tailored for data specifications have not yet been proposed.

5.2 Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation [4]

This paper proposes a benchmark mutation methodology that transforms formal problem descriptions from existing software engineering datasets into realistic user queries. Current evaluation benchmarks for SE agents, rely on GitHub issues that do not accurately reflect the ways developers communicate with chat-based coding assistants in practice. The authors extract common communication patterns from real developer conversations with coding assistants and use them to mutate

formal benchmarks systematically. More broadly from a AI Engineering perspective, this approach aligns system evaluation with actual operational conditions. The aim is to improve benchmark quality for AI systems.

I think this paper shows the danger of defining data requirements based on idealized scenarios rather than actual user behavior. The authors show that current benchmarks for code assistants are based off of well-structured Github data, which is entirely different from how an actual developer interacts with an LLM when working. This means data requirements must carefully define the expected format and ambiguity of real-world inputs. The paper also provides a practical method for eliciting data requirements through telemetry analysis. This is similar to previous research where one extracts requirements from APP reviews. Extracted patterns can become the data requirements that the AI system must be designed to handle.

A straightforward AI project for this would be an AI coding assistant deployed across a large software company. The machine learning component is an interactive coding agent that interacts with repository files and generates code edits based on natural language chat inputs from developers. The approach proposed by this paper can be directly used to evaluate and improve the system. The engineering team can analyze their own internal telemetry to see how their developers actually talk to the assistant. They can extract recurring communication patterns and use these templates to rewrite their internal test suites into realistic chat prompts. Testing the coding agent against these mutated queries allows the team to accurately measure how the system handles daily developer workflows. From my own research, I think I need to consider that prompts and their contextual payloads function as runtime input data. This requires defining the contextual elements that must be aggregated into the prompt while adhering to possible token budget limits. Data requirements must also establish specifications for prompt quality, formatting, and safety. Requirements need to also cover mandatory pre-processing constraints, like removing private customer data before the prompt is constructed.

With how popular LLM based systems are getting, I think I might need to shift my focus of data requirements from static training datasets to the data provided at runtime. While these models consume massive amounts of data during training, their success in a software product depends on the prompt. Therefore, prompt assembly, context retrieval, and user inputs need to be treated as data pipelines that need engineering rules as well.

6 Evidence, originality, and GenAI-use transparency

All of my references have been provided below in the References section. They are all peer reviewed papers, from reputable sources (journals, conferences). For the CAIN'26 papers I used Gemini to generate the bibtex citations because they are too new to show up in google scholar, which I usually use to get citations in bibtex form. I mainly used papers that were provided during the lecture. For Goal Models, I used google scholar with the search phrases "goal modeling AI system" and "goal modeling machine learning". The references used in the introduction section are mostly from my own literature review (which was the first paper I wrote in my PhD). I also used Gemini to help me with the latex formatting, i.e., adding the .bib citations into the .tex file and not a separate .bib file.

References

[1] Shraddha Barke, Michael B James, and Nadia Polikarpova. "Grounded copilot: How programmers interact with code-generating models". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 85–111.

[2] Justus Bogner, Roberto Verdecchia, and Ilias Gerostathopoulos. "Characterizing technical debt and antipatterns in AI-based systems: A systematic mapping study". In: *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*. IEEE. 2021, pp. 64–73.

[3] Alessio Ferrari and Paola Spoletini. "Formal requirements engineering and large language models: A two-way roadmap". In: *Information and Software Technology* 181 (2025), p. 107697.

[4] Spandan Garg, Benjamin Steenhoek, and Yufan Huang. "Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation". In: *Proceedings of the 5th International Conference on AI Engineering – Software Engineering for AI (CAIN 2026)*. ACM, 2026.

[5] Christoph Gröger. "There is no AI without data". In: *Communications of the ACM* 64.11 (2021), pp. 98–108.

[6] Dorien Herremans. "aiSTROM–A roadmap for developing a successful AI strategy". In: *IEEE Access* 9 (2021).

[7] Oz Kilic et al. "RegDriftKit: A Toolkit for Generating Data and Benchmarking Drift Detection in Regression Tasks". In: *Proceedings of the 5th International Conference on AI Engineering - Software Engineering for AI (CAIN 2026)*. Rio de Janeiro, Brazil, Apr. 2026.

[8] Tong Li et al. "An interactive and AI-enhanced framework for semi-automatically generating iStar goal models". In: *Automated Software Engineering* 33.2 (2026), p. 41.

[9] Nadia Nahar et al. "Collaboration challenges in building ML-enabled systems: Communication, documentation, engineering, and process". In: *Proceedings of the 44th international conference on software engineering*. 2022, pp. 413–425. URL: https://doi.org/10.1145/3510003.3510209.

[10] David Sculley et al. "Hidden technical debt in machine learning systems". In: *Advances in neural information processing systems* 28 (2015).

[11] Melika Soleimani, Ali Intezari, and David J Pauleen. "Mitigating cognitive biases in developing ai-assisted recruitment systems: A knowledge-sharing approach". In: *International Journal of Knowledge Management (IJKM)* 18.1 (2022), pp. 1–18.

[12] Melika Soleimani et al. "Cognitive biases in developing biased Artificial Intelligence recruitment system". In: *Hawaii International Conference on System Sciences*. 2021. URL: https://api.semanticscholar.org/CorpusID:232414455.

[13] Elizamary de Souza Nascimento et al. "Understanding development process of machine learning systems: Challenges and solutions". In: *2019 acm/IEEE international symposium on empirical software engineering and measurement (esem)*. IEEE. 2019, pp. 1–6. URL: https://doi.org/10.1109/ESEM.2019.8870157.

[14] Kuldar Taveter and Mohamad Gharib. "Eliciting and modelling human values for human-centric artificial intelligence systems with motivational goal models". In: *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*. IEEE. 2025, pp. 439–441.

[15] Axel Van Lamsweerde et al. "Goal-oriented requirements engineering: A guided tour." In: *re* 1 (2001), p. 249.

[16] Asma Yamani et al. “DRGM: Data Requirement Goal Modeling for ML-Based Systems”. In: *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*. 2025, pp. 62–73. DOI: 10.1109/REW66121.2025.00013.